

科目：データベース演習

# 第13回：SQL応用：アクセスログのセッション分析 (入門)

講師：鈴木源吾

## 前回の復習

1. サブクエリ: 問合せの結果を問合せで利用する
2. ウィンドウ関数でグループ全体を対象にした計算をする

# 今回のトピック

1. セッション分析：アクセス間隔を求める
  - LAGウィンドウ関数の利用
2. セッション分析：セッション識別
  - CASE式の利用

その他の資料：第10章

topic 1

**セッション分析：アクセス間隔を求める**

## 今回の内容について

- 今回の内容は、その他の資料「10年戦えるデータ分析入門」の第10章の、アクセスログのセッション分析の、最初の部分を解説する
- やや中途半端で終わるため、興味のある人は、続きを読んで見ることに

# セッション分析とは

- セッションとは、ユーザーがウェブサイトを利用しているひとまとまりの時間、あるいは、その時間に発生するウェブサイトへのアクセス
- セッションの例
  - 商品を見る→他の商品も見比べて比較する→購入する
  - （Googleの検索結果から飛んできて）何かのページを見て終わり
- セッション分析：セッションの特徴を分析すること
  - セッションが長い・短い
  - どこからこのサイトへ来たか
  - このサイト内でどんな行動をとったか

# セッション分析の例

その他の資料「10年戦えるデータ分析入門」では、

## 顧客が商品検索をした後にカートに商品を入れる率

を求めている。これはコンバージョン率と呼ばれる

- サイトにおいて何かを達成することを「コンバージョン（顧客転換）」と呼ぶため（マーケティング用語）。この場合は、カートに商品を入れることを達成

これを求めるために、「パスパターンの頻度分析」を行っている

- パスパターンとは、Webサイトにおけるユーザーの移動のパターン
- 頻度分析なので、頻繁に起こるパスパターンを調べる分析

# セッションナイズ

しかし、今回は、セッション分析の入り口の**セッションナイズ**のみ解説する

セッションナイズ：アクセスが所属するセッションを決めること

- ユーザーは様々なアクセスを行う
- ある1人のユーザーの「一連の」アクセスがセッション

→ 今回は、**同一のユーザーの連続するアクセスで、アクセス間隔が15分未満である一連のアクセスを1つのセッションとする**

- アクセス間隔とは、時系列で隣接するアクセスのアクセス時刻の差とする
- このやり方は、時間で単純に判断するシンプルな方式
- 他にもセッションナイズの方式はいくつかある

# アクセス間隔によるセッションナイズ

 alt

# アクセス間隔を求めよう

access\_logテーブルのrequest\_timeカラム（アクセスのあった時刻）を利用する  
まずは、「連続する」2つの行のrequest\_timeの差を求めたい

- これは、その2つのアクセス時刻の間隔を表す（同一ユーザとは限らない）
- この差を求めるには、一つ前（または一つ後）の行をとる機能がほしい

 alt

# 1つ前の行の値をとる：LAGウィンドウ関数

LAGウィンドウ関数は、1つ前の行の値を取得できる  
まずは、以下のシンプルな例で確認してみよう

 alt

# LAGウィンドウ関数を使ってアクセス間隔を求める

前ページから，連続する2つの行のrequest\_timeの差は，以下で求められる

```
request_time - LAG(request_time)
```

さらに，同一ユーザの連続するアクセスでこれを利用するには，

- セッションは，異なるユーザが混じってはいけなから，アクセス間隔も，同一のユーザのアクセス内で決める必要がある
- アクセスログは，request\_timeカラムの値の順番に並んでいるとは限らない（RDBMSは行の順序は保証されない）．よって，ソートが必要である

# アクセス間隔を求めるSQL文

 alt

# 例題1

customersテーブルは顧客の情報である、顧客のID (customer\_idカラム)、年齢 (customer\_ageカラム) などを管理している。顧客の年齢にギャップがどの程度あるかを調べたい。年齢のギャップとは、年齢が小さい順に顧客を並べたときの隣接する2人の顧客の年齢の差を意味することとする。LAGウィンドウ関数を用いることにより、顧客のID、顧客の年齢、年齢のギャップ (AS句を用いage\_gapという名前にせよ) の組を、age\_gapの大きい順に並べ、最大10行のみ求めるSQL文を作成せよ



## 例題1のSQL文の実行例

- age\_gapがNULLである行（customer\_ageが最も小さい顧客に関する行）が先頭に来るが、この行は排除しなくとも良いこととする。以下の結果から年齢のギャップは最大1であることがわかる。

 alt

## 演習 課題6-3

itemsテーブルは商品の情報である、商品のID (item\_idカラム)、名前 (item\_nameカラム) 価格 (item\_priceカラム) などを管理している。商品の価格にギャップがどの程度あるかを調べたい。価格のギャップとは、価格が安い順 (安い商品→高い商品の順) に商品を並べたときの隣接する2つの商品の価格の差を意味することとする。LAGウィンドウ関数を用いることにより、商品のID、商品の名前、商品の価格、価格のギャップ (AS句を用いprice\_gapという名前にせよ) の組を、price\_gapの大きい順に並べ、最大10行のみ求めるSQL文を作成せよ

## 演習 課題6-3のSQL文の実行例

- price\_gapがNULLである行（item\_priceが最も小さい商品に関する行）が先頭に来るが、この行は排除しなくとも良いこととする。以下の結果から価格のギャップは最大7700であることがわかる。

 alt

topic 2

## セッション分析：セッション識別

# セッション識別

セッションの定義は,

「同一のユーザーの連続するアクセスで、アクセス間隔が15分未満である一連のアクセス」

だった. topic 1で、アクセス間隔を求めることができたから、さらにセッションを識別する（決める）ためには、アクセス間隔が15分未満であるような、アクセス（の組）を見つける必要がある

## セッション識別で欲しい結果

アクセスログに対して、アクセス間隔を求め、以下のようにセッションを表す番号を付与できればよい

 alt

# セッション番号の求め方

以下の2つの組合せで、セッション番号は求められる

- セッションの先頭→1, 先頭でない→0というフラグを求める
- セッション先頭フラグの累積和が、セッション番号

 alt

# セッション番号の求め方

## セッション先頭フラグの求め方

- アクセス間隔が15分以上→1
- アクセス間隔が15分未満→0

とすればよい → CASE式を用いる（後で説明する）

## セッション番号の求め方

- セッション先頭フラグの累積和 → SUMウィンドウ関数を使う。
  - OVER句で「ROWS BETWEEN 前側の行 AND 後側の行」と指定できる機能を利用し、現在の行の前のすべてを合計する
  - 今回は説明しない。興味のある人は、「10年使えるデータ分析」を参照
  - 本トピックの最後に、参考としてセッション番号を求めるSQL文を掲載する

# SQLにおけるCASE式

CASE式とは、カラムの値の一致や、条件式の結果により、返却する値を変えるときに使う式である。返却値は新しく定義したカラムの値として利用される

→ プログラミング言語における条件分岐と同等

CASE式に、単純CASE式と検索CASE式がある

- 単純CASE式：カラムの値により分岐し、返却する値を変える
- 検索CASE式：条件式の結果により分岐し、返却する値を変える

# 単純CASE式

単純CASE式の文法は一般には以下となる

- 条件の対象となるカラムの値が、 WHEN句のカラムの値に一致するかを上から調べる
- 最初に一致したときに、 そのTHEN句にある返却値が返される
- すべての条件に一致しない場合、 ELSE句にある返却値が返される

```
CASE 条件の対象となるカラム名
  WHEN カラムの値1 THEN 返却値1
  WHEN カラムの値2 THEN 返却値2
  WHEN カラムの値3 THEN 返却値3
  ...
  ELSE 返却値n
END
```

## 単純CASE式の例

例えば、customersテーブルで、customer\_name カラムの値とあわせて、customer\_genderカラムの値がFだったら女性、Mだったら男性、それ以外ならその他の値を返すSQL文は、以下となる

```
SELECT customer_name,  
       CASE customer_gender  
         WHEN 'F' THEN '女性'  
         WHEN 'M' THEN '男性'  
         ELSE 'その他'  
       END  
FROM customers;
```

# 検索CASE式

検索CASE式の文法は一般には以下となる

- WHEN句の条件式が真になるかを上から調べる
- 最初に真になったときに、そのTHEN句にある返却値が返される

```
CASE  
  WHEN 条件式1 THEN 返却値1  
  WHEN 条件式2 THEN 返却値2  
  WHEN 条件式3 THEN 返却値3  
  ...  
  ELSE 返却値n  
END
```

# 検索CASE式の利用例

例えば、itemsテーブルを用いて、商品に関して、item\_nameカラムの値と合わせて、価格（item\_priceカラム）が1万円より高かったら「高い！」、価格が3000円より安かったら「安い！」、そうでなかったら「そこそこ」を、カラムの別名に「評価」とつけて返すSQL文は以下となる

```
SELECT item_name, item_price,  
       CASE  
           WHEN item_price > 10000 THEN '高い！'  
           WHEN item_price < 3000 THEN '安い！'  
           ELSE 'そこそこ'  
       END AS 評価  
FROM items;
```

# セッション先頭フラグを求めるSQL文

- 最初の行だけは、アクセス間隔を求めるとnullになる（一つ前のタプルがないから）。この行は最初のセッションの先頭なので、先頭フラグを1とする
- その後は、アクセス間隔が15分以上であれば1として、それ以外を0とすればよい  
→CASE式が利用できる！



# セッション先頭フラグを求めるSQL文

別名access\_intervalを、SELECT句内で参照できないため冗長な記述要  
時刻の差はinterval型となるため、15分を"interval '15 minute'"と記述する必要あり

```
SELECT customer_id, request_time, request_time - LAG(request_time) OVER (  
    PARTITION BY customer_id  
    ORDER BY request_time  
    ) AS access_interval,  
    CASE  
        WHEN request_time - LAG(request_time) OVER (  
            PARTITION BY customer_id  
            ORDER BY request_time  
            ) is null THEN 1  
        WHEN request_time - LAG(request_time) OVER (  
            PARTITION BY customer_id  
            ORDER BY request_time  
            ) >= interval '15 minute' THEN 1  
        ELSE 0  
    END AS session_start_flag  
FROM access_log LIMIT 100;
```

## 参考：セッション番号を求めるSQL文

```
SELECT customer_id, request_time, session_start_flag, SUM(session_start_flag) OVER (
    PARTITION BY customer_id
    ORDER BY request_time
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS session_id
FROM (
    SELECT customer_id, request_time, request_time - LAG(request_time) OVER (
        PARTITION BY customer_id
        ORDER BY request_time
    ) AS access_interval,
    CASE
        WHEN request_time - LAG(request_time) OVER (
            PARTITION BY customer_id
            ORDER BY request_time
        ) is null THEN 1
        WHEN request_time - LAG(request_time) OVER (
            PARTITION BY customer_id
            ORDER BY request_time
        ) >= interval '15 minute' THEN 1
        ELSE 0
    END AS session_start_flag
    FROM access_log
);
```

## 参考：セッション番号を求めるSQL文の実行結果

間隔が15分未満のアクセスは同一セッションとなっている（結果は一部のみ）

 alt

## 例題2

顧客を表すcustomersテーブルを用いて、氏名（customer\_nameカラム）と、年齢（customer\_ageカラム）その人が属する年齢グループ（別名をage\_groupとせよ）の組を、最大10行、返却するSQL文を作成せよ。年齢グループの値は、30歳未満：若者、30歳以上50歳未満：中年、50歳以上：老人、という値をとることとする。検索結果例を以下に示す



alt

## 演習 課題6-4

顧客を表すcustomersテーブルを用いて、顧客のID (customer\_idカラム)、氏名 (customer\_nameカラム) と、居住地 (customer\_locationカラム) その人が新潟県に住んでいるかどうかのフラグ (別名をniigata\_flagとせよ) の組を、最大10行、返却するSQL文を作成せよ。niigata\_flagの値は、居住地が新潟県である場合に1をとり、それ以外の都道府県である場合は0をとることとする。検索結果例を以下に示す



alt